

Multi-threaded Data Processing System in Java and Go

Xhon Pelushi

University of the Cumberland

MSCS-632-M30: Advanced Programming Languages

Dr. Jay Thom

July 28, 2026

Overview

This report describes a multi-threaded data processing system implemented independently in Java and Go. The system models a ride-trip fare processor: twelve trip records are placed on a shared task queue, four worker threads (or goroutines) retrieve those tasks in parallel, each worker simulates computational work with a short delay, and completed fare results are written to a shared in-memory list and a CSV output file. Both implementations synchronize access to shared resources, log worker lifecycle events and errors, and shut down cleanly once the queue is drained. The goal is to compare Java's shared-memory concurrency model with Go's communicating sequential processes style while meeting the same functional requirements (Goetz et al., 2006; Donovan & Kernighan, 2015).

Java Concurrency and Exception Handling

The Java implementation centers on a `TaskQueue` class whose `addTask()` and `getTask()` methods are guarded by a `ReentrantLock` and a `Condition`. Workers that find the queue empty call `await()` on the condition and are woken when a producer enqueues a task or when `close()` signals that no further work will arrive. Using an explicit lock and condition avoids busy-waiting and prevents the classic mistake of checking the empty-queue predicate outside a synchronized region (Goetz et al., 2006). Worker threads themselves are managed through `Executors.newFixedThreadPool(4)`, which reuses a fixed set of threads and returns a `Future` for each `Worker Callable` so the main thread can wait for completion without joining threads manually.

A separate `ResultStore` holds the shared results list and a `BufferedWriter` to the CSV file. Every write acquires the same `ReentrantLock`, so concurrent workers cannot interleave CSV lines or corrupt the `ArrayList`. Exception handling follows Java's checked-exception model: `getTask()` declares `InterruptedException` and workers catch it, restore the interrupt flag, and exit; `ResultStore` catches `IOException` on write failures and logs them without crashing sibling

workers; the main method wraps file construction in try-with-resources so the writer is closed even if an error occurs, and the executor is shut down in a finally block with `awaitTermination()` followed by `shutdownNow()` if workers do not finish in time. Logging records when each worker starts, completes a trip, exits on an empty closed queue, or encounters an error.

Go Concurrency and Error Handling

The Go implementation uses a buffered channel of Task values as the shared queue. The main goroutine sends twelve trip records into the channel and then closes it; four worker goroutines range over the channel until it is drained and closed, which is Go's idiomatic safe-termination pattern (Donovan & Kernighan, 2015). Because channel send and receive are already concurrency-safe, the program does not need an explicit mutex around the task queue itself. A `sync.WaitGroup` tracks outstanding workers so main blocks until every goroutine has called `Done()`.

Shared output still requires synchronization: `ResultStore` protects its results slice and CSV writer with a `sync.Mutex`, and `Add()` uses `defer mu.Unlock()` so the lock is released even when a write error is returned. Go's error model is explicit rather than exception-based. `NewResultStore` and `Add` return error values that callers check immediately; a failure to open the results file is fatal via `log.Fatalf`, while a per-row write failure is logged by the worker and processing continues for remaining tasks. The results file is closed in a deferred function in main so cleanup runs on both success and failure paths. Each worker logs start, per-task completion, write errors, and a final processed count.

Concurrency Model Differences

The two languages embody different concurrency philosophies. Java treats threads as schedulable units that share heap memory, so correctness depends on the programmer applying locks, conditions, and careful shutdown of an `ExecutorService` around every shared structure. That model is powerful and familiar, but it places the burden of avoiding race conditions and

deadlocks on explicit synchronization discipline (Goetz et al., 2006). Go favors communication over shared mutable state: channels carry tasks between goroutines, closing a channel broadcasts completion, and the language runtime multiplexes many goroutines onto fewer OS threads (Donovan & Kernighan, 2015). Mutexes still appear where shared memory is unavoidable, such as the results file, but the primary coordination mechanism is the channel rather than a lock-protected LinkedList.

Error handling diverges in a related way. Java's try-catch and try-with-resources concentrate cleanup and recovery around exception types such as InterruptedException and IOException. Go returns errors as ordinary values and uses defer for cleanup, which makes failure paths visible at each call site instead of in a distant catch block. In practice both programs processed all twelve trips with four workers, wrote matching fare results to CSV, and terminated without deadlock; the difference is which tool each language makes the default for expressing that coordination.

Conclusion

Implementing the same multi-threaded data processing system in Java and Go showed that shared queues, worker pools, synchronized result writing, logging, and orderly shutdown can be achieved in both languages, but the idioms differ. Java relies on ReentrantLock, Condition, and Executors with try-catch around interruptions and I/O. Go relies on goroutines, channels, WaitGroup, mutex-guarded file writes, and defer-based cleanup with explicit error checks. Choosing between them is less about capability and more about whether a project prefers shared-memory locking or message-passing coordination.

Source Code

The complete source code for both implementations is available in the public GitHub repository: <https://github.com/xhon-pelushi/MSCS-632-Advanced-Programming-Languages/tree/main/Assignment-6>

References

- Donovan, A. A. A., & Kernighan, B. W. (2015). The Go programming language. Addison-Wesley.
- Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., & Lea, D. (2006). Java concurrency in practice. Addison-Wesley.

Appendix: Screenshots

```
xadmin@ubuntu: ~/Assignment-6/java
xadmin@ubuntu:~/Assignment-6/java$ javac *.java && java DataProcessingSystem
INFO: Starting Java data processing system with 4 workers
INFO: Worker-3 started
INFO: Worker-2 started
INFO: Worker-4 started
INFO: Worker-1 started
INFO: Enqueued 12 trip tasks
INFO: Task queue closed; remaining size=8
INFO: Worker-3 completed Trip#101 [Alice, 3.2 mi, 12 min] -> fare $12.30
INFO: Worker-2 completed Trip#102 [Bob, 8.5 mi, 24 min] -> fare $25.78
INFO: Worker-4 completed Trip#103 [Carla, 1.1 mi, 6 min] -> fare $6.53
INFO: Worker-1 completed Trip#104 [Diego, 12.0 mi, 35 min] -> fare $35.75
INFO: Worker-3 completed Trip#105 [Elena, 5.4 mi, 18 min] -> fare $18.25
INFO: Worker-2 completed Trip#106 [Farid, 2.0 mi, 9 min] -> fare $9.15
INFO: Worker-4 completed Trip#107 [Gina, 15.7 mi, 42 min] -> fare $44.68
INFO: Worker-1 completed Trip#108 [Hiro, 4.3 mi, 15 min] -> fare $15.27
INFO: Worker-3 completed Trip#109 [Ivy, 6.8 mi, 21 min] -> fare $21.75
INFO: Worker-3 exiting: queue closed and empty
INFO: Worker-4 completed Trip#110 [Jamal, 9.2 mi, 28 min] -> fare $28.40
INFO: Worker-4 exiting: queue closed and empty
INFO: Worker-2 completed Trip#111 [Kara, 0.8 mi, 4 min] -> fare $5.30
INFO: Worker-3 finished; processed=3
INFO: Worker-2 exiting: queue closed and empty
INFO: Worker-4 finished; processed=3
INFO: Worker-2 finished; processed=3
INFO: Worker-1 completed Trip#112 [Leo, 11.3 mi, 31 min] -> fare $33.13
INFO: Worker-1 exiting: queue closed and empty
INFO: Worker-1 finished; processed=3
INFO: All workers finished. Total trips processed=12, results written=12
... (log truncated) ...
105,Elena,5.4,18,18.25,Worker-3
106,Farid,2.0,9,9.15,Worker-2
107,Gina,15.7,42,44.68,Worker-4
108,Hiro,4.3,15,15.27,Worker-1
109,Ivy,6.8,21,21.75,Worker-3
110,Jamal,9.2,28,28.40,Worker-4
111,Kara,0.8,4,5.30,Worker-2
112,Leo,11.3,31,33.13,Worker-1
INFO: Result file closed: /home/xadmin/Code/School/2026 Summer - Advanced Programming Languages (MSCS-632-M
INFO: Executor shut down; system terminated safely
xadmin@ubuntu:~/Assignment-6/java$
```

Figure 1. Java run: four workers process twelve trip tasks and write results `_java.csv`.

```

xadmin@ubuntu: ~/Assignment-6/java
xadmin@ubuntu:~/Assignment-6/java$ sed -n '18,55p;28,55p' TaskQueue.java Worker.java
// TaskQueue.java - ReentrantLock shared queue
18 |     private boolean closed = false;
19 |
20 |     public void addTask(Task task) {
21 |         lock.lock();
22 |         try {
23 |             if (closed) {
24 |                 throw new IllegalStateException("Cannot add tasks after the queue is closed");
25 |             }
26 |             queue.addLast(task);
27 |             LOG.info("Enqueued " + task);
28 |             notEmpty.signal();
29 |         } finally {
30 |             lock.unlock();
31 |         }
32 |     }
33 |
34 |     /**
35 |      * Blocks until a task is available, or returns empty if the queue is closed
36 |      * and drained (safe worker termination signal).
37 |      */
38 |     public Optional<Task> getTask() throws InterruptedException {
39 |         lock.lock();
40 |         try {
41 |             while (queue.isEmpty() && !closed) {
42 |                 notEmpty.await();
43 |             }
44 |             if (queue.isEmpty()) {
45 |                 return Optional.empty();
46 |             }
47 |             Task task = queue.removeFirst();
48 |             LOG.info("Dequeued " + task);
49 |             return Optional.of(task);
50 |         } finally {
51 |             lock.unlock();
52 |         }
53 |     }
54 |
55 |     /** Mark the queue closed and wake all waiting workers so they can exit. */

// Worker.java - Callable worker with try/catch
28 |     Thread.currentThread().setName(name);
29 |     LOG.info(name + " started");
30 |     int processed = 0;
31 |
32 |     try {
33 |         while (true) {
34 |             Optional<Task> maybeTask;
35 |             try {
36 |                 maybeTask = queue.getTask();
37 |             } catch (InterruptedException e) {
38 |                 Thread.currentThread().interrupt();
39 |                 LOG.log(Level.WARNING, name + " interrupted while waiting for a task", e);
40 |                 break;
41 |             }
42 |
43 |             if (maybeTask.isEmpty()) {
44 |                 LOG.info(name + " exiting: queue closed and empty");
45 |                 break;
46 |             }
47 |
48 |             Task task = maybeTask.get();
49 |             try {
50 |                 Thread.sleep(processingDelayMs);

```

Figure 2. Java source excerpt: *ReentrantLock TaskQueue* and *Worker* try/catch processing loop.

```
xadmin@ubuntu: ~/Assignment-6/go
xadmin@ubuntu:~/Assignment-6/go$ go run .
Starting Go data processing system with 4 workers
Worker-1 started
Enqueued 12 trip tasks
Worker-4 started
Worker-2 started
Worker-3 started
Worker-3 completed Trip#104 [Diego, 12.0 mi, 35 min] -> fare $35.75
Worker-4 completed Trip#102 [Bob, 8.5 mi, 24 min] -> fare $25.78
Worker-1 completed Trip#101 [Alice, 3.2 mi, 12 min] -> fare $12.30
Worker-2 completed Trip#103 [Carla, 1.1 mi, 6 min] -> fare $6.53
Worker-2 completed Trip#108 [Hiro, 4.3 mi, 15 min] -> fare $15.27
Worker-3 completed Trip#105 [Elena, 5.4 mi, 18 min] -> fare $18.25
Worker-4 completed Trip#106 [Farid, 2.0 mi, 9 min] -> fare $9.15
Worker-1 completed Trip#107 [Gina, 15.7 mi, 42 min] -> fare $44.68
Worker-1 completed Trip#112 [Leo, 11.3 mi, 31 min] -> fare $33.13
Worker-1 finished; processed=3
Worker-3 completed Trip#110 [Jamal, 9.2 mi, 28 min] -> fare $28.40
Worker-3 finished; processed=3
Worker-2 completed Trip#109 [Ivy, 6.8 mi, 21 min] -> fare $21.75
Worker-2 finished; processed=3
Worker-4 completed Trip#111 [Kara, 0.8 mi, 4 min] -> fare $5.30
Worker-4 finished; processed=3
All workers finished. Results written=12

=== Go Multi-threaded Data Processing System ===
Workers: 4
Trips submitted: 12
Trips processed: 12
... (log truncated) ...
103,Carla,1.1,6,6.53,Worker-2
108,Hiro,4.3,15,15.27,Worker-2
107,Gina,15.7,42,44.68,Worker-1
105,Elena,5.4,18,18.25,Worker-3
106,Farid,2.0,9,9.15,Worker-4
112,Leo,11.3,31,33.13,Worker-1
111,Kara,0.8,4,5.30,Worker-4
110,Jamal,9.2,28,28.40,Worker-3
109,Ivy,6.8,21,21.75,Worker-2
Result file closed: results_go.csv
xadmin@ubuntu:~/Assignment-6/go$
```

Figure 3. Go run: four goroutines drain a task channel and write *results_go.csv*.


```

xadmin@ubuntu: ~/Assignment-6/go
xadmin@ubuntu:~/Assignment-6/go$ sed -n '106,132p;172,188p' main.go
// main.go - channel queue + goroutine workers
106 | func worker(id int, tasks <-chan Task, results *ResultStore, wg *sync.WaitGroup) {
107 |     defer wg.Done()
108 |     name := fmt.Sprintf("Worker-%d", id)
109 |     log.Printf("%s started", name)
110 |     processed := 0
111 |
112 |     for task := range tasks {
113 |         time.Sleep(processingDelay)
114 |         fare := task.ComputeFare()
115 |         row := []string{
116 |             fmt.Sprintf("%d", task.TripID),
117 |             task.RiderName,
118 |             fmt.Sprintf("%.1f", task.DistanceMiles),
119 |             fmt.Sprintf("%d", task.DurationMinutes),
120 |             fmt.Sprintf("%.2f", fare),
121 |             name,
122 |         }
123 |         if err := results.Add(row); err != nil {
124 |             log.Printf("%s error writing result for %s: %v", name, task, err)
125 |             continue
126 |         }
127 |         processed++
128 |         log.Printf("%s completed %s -> fare %.2f", name, task, fare)
129 |     }
130 |
131 |     log.Printf("%s finished; processed=%d", name, processed)
132 | }

// main - spawn workers, enqueue tasks, close channel
172 | // Buffered channel acts as the shared, concurrency-safe task queue.
173 | tasks := make(chan Task, 16)
174 |
175 | var wg sync.WaitGroup
176 | log.Printf("Starting Go data processing system with %d workers", workerCount)
177 | for i := 1; i <= workerCount; i++ {
178 |     wg.Add(1)
179 |     go worker(i, tasks, results, &wg)
180 | }
181 |
182 | trips := sampleTrips()
183 | for _, t := range trips {
184 |     log.Printf("Enqueued %s", t)
185 |     tasks <- t
186 | }
187 | log.Printf("Enqueued %d trip tasks", len(trips))
188 | close(tasks) // safe termination: workers exit when channel drains
xadmin@ubuntu:~/Assignment-6/go$

```

Figure 4. Go source excerpt: channel-based workers, *WaitGroup*, and deferred result-file close.